Yancheng (Jason) Liu

Research Statement

MRes in AI and Machine Learning, Imperial College London
j.liu26@imperial.ac.uk | setsuna113.github.io

I study how LLM agents should manage their memory, the KV cache and other inference state, when each decision about what to keep, in what form and where is made before its consequences are visible. Long-horizon agents face these decisions at every step because GPU memory is fixed, and the cost of each decision appears only later. History compressed away now can cause a wrong action turns later, which changes the environment before the error is noticed, or it has to be recomputed when the agent resumes.

I came to this question through my own projects. In *Racer* (co-first author, under review at ICLR 2027), a KV cache management framework for multi-turn agents, each compressed block of history looked harmless on its own, and the errors surfaced turns later in the agent's actions, so Racer predicts the risk of each drafted action before it executes and, when needed, regenerates it from the original evidence. In *ShapeFlow*, measurement showed that a deep-research agent's cost lay in the form of its intermediate evidence. The better form depends on what each task retrieves and how busy the engine is, so I deferred that choice to a runtime scheduler. For my MRes at Imperial, I am working on the inference state of hybrid and sparse-attention models: what to keep, in which recoverable form and where in the memory hierarchy, so that an agent can resume cheaply after a pause such as a tool call. Three questions follow.

1. *Can the consequence of a memory decision be predicted before the agent acts?* Racer's predictor reads the prefill hidden state and the draft's average NLL, and its label is whether the turn fails when the draft runs without recovery, so it learns when skipping recovery breaks the agent. I want to move this prediction earlier, to the moment a block is compressed or evicted, so that the memory policy itself sees the risk, not only a later repair step.
2. *Which recoverable form should each part of the history take?* A memory decision is cheap to get wrong if the discarded information can be brought back. Racer offloads the raw history to CPU memory and restores the relevant text when a draft looks risky; ShapeFlow passes span IDs that are expanded into source text only when the report is written. The right form depends on how likely a piece is to be needed and how busy the engine is when it is, which are known only at runtime, so I want the serving system to choose form and placement together.
3. *How can an agent learn what to keep from delayed outcomes?* In Racer I trained our group's compressor with the next correct tool call as its target, a one-step signal. Whether a piece of history mattered is settled by the outcome of the episode, which requires training on the model's own rollouts, where errors compound across turns, and assigning credit to memory decisions made many steps earlier.

All three rest on a mechanistic question: which layers and heads read a piece of memory back later, and whether the hidden state signals that needed information is missing. If only some layers read old history, only those layers need an exact KV cache; the hybrid models of my MRes already keep one in only some layers. I have worked at this level: Racer's risk predictor reads the prefill hidden state, and in a Cambridge UROP I proved that an MC-dropout error signal measures the margin fluctuation that dropout induces and traced it to the lower half of the blocks.

My work spans both sides: compression training, failure prediction and mechanistic analysis on the model side; on the system side, Racer's concurrent SGLang runtime with session-owned compressed KV and the ShapeFlow scheduler beside an unmodified vLLM. In a PhD I want to make agent memory one line of work, from the policy that decides what to keep to the system that keeps it.